M365 & Azure Incident Response & Escalation Runbook

Implementation Solution Guide

Version 1.0.0 · 2026-06-27

Contents

Executive Summary — M365 & Azure Incident Response & Escalation Runbook 5
The business problem . . . 5
What this package delivers . . . 5
The phase model . . . 5
Outcomes . . . 5
Honest scope . . . 6
Investment . . . 6
1. Solution Overview — M365 & Azure Incident Response & Escalation Runbook 6
What this solution is . . . 6
The phase model . . . 6
The named artifacts . . . 7
The per-workload diagnostic playbooks . . . 7
How it works (the flow) . . . 8
Benefits . . . 8
Honest limitations . . . 9
The standard runbook model . . . 9
2. Requirements — M365 & Azure Incident Response & Escalation Runbook 9
R1. Responder identities with the right M365 / Azure roles (least privilege) . . . 9
R2. A defined responder group + on-call rota . . . 10
R3. Access to Microsoft 365 Service Health . . . 10
R4. Entra ID sign-in & audit log access . . . 10
R5. Exchange Online message trace + transport access . . . 11
R6. Defender / security signal access (for security-driven incidents) . . . 11
R7. Azure Monitor / Resource Health access . . . 11
R8. A Microsoft support plan (and knowledge of its severity rules) . . . 11
R9. A ticketing / PSA system as the system of record . . . 12
R10. Defined on-call communication channels + a client status method . . . 12
R11. Defined response-time / SLA targets . . . 12
R12. Change-control & rollback posture for remediations . . . 13
3. Implementation — M365 & Azure Incident Response & Escalation Runbook 13
Part A — Stand up the runbook (one-time) 13
A1. Build the Severity & Impact Matrix – Incident – Triage . . . 13
A2. Build the Incident Record Template – Incident – Lifecycle . . . 14
A3. Build the Roles & On-Call Sheet – Incident – Response . . . 15
A4. Build the Client Comms Templates – Incident – Communicate . . . 15

A5. Build the Microsoft Escalation Sheet – Incident – Escalate . . . 15
A6. Build the Diagnostic Playbooks – Incident – Diagnose . . . 16
A7. Build the RCA Template – Incident – Review and the Problem Record – Incident – Permanent Fix . . . 18
A8. Provision responder access (R1, R2) . . . 18
Part B — The per-incident runbook (every incident) 18
Phase 1 — Detect . . . 18
Phase 2 — Triage . . . 19
Phase 3 — Communicate . . . 19
Phase 4 — Diagnose . . . 19
Phase 5 — Resolve . . . 19
Phase 6 — Escalate to Microsoft (when needed) . . . 20
Phase 7 — RCA → Permanent Fix . . . 20
5. Testing — M365 & Azure Incident Response & Escalation Runbook 20
T1. Responder readiness — roles & access *(gate)* . . . 20
T2. Support-plan & escalation readiness *(gate)* . . . 21
T3. Triage correctness *(gate)* . . . 21
T4. Service Health discrimination *(gate)* . . . 21
T5. Communication on cadence *(gate)* . . . 21
T6. Diagnosis reaches root cause *(gate)* . . . 22
T7. Remediation safety *(gate)* . . . 22
T8. RCA produced *(gate)* . . . 22
T9. Permanent fix logged *(gate — overall)* . . . 22
Acceptance gate (overall) . . . 23
6. Troubleshooting — M365 & Azure Incident Response & Escalation Runbook 23
Access & visibility (Detect / Diagnose) . . . 23
Service Health discrimination (Triage) . . . 23
Triage & roles . . . 23
Communication . . . 24
Escalation to Microsoft . . . 24
Remediation . . . 24
RCA → Permanent Fix loop . . . 24
7. Best Practices — M365 & Azure Incident Response & Escalation Runbook 25
Triage & prioritization . . . 25
Roles & coordination . . . 25
Client communication . . . 25
Diagnosis . . . 26

Escalation to Microsoft 26
Remediation safety 26
RCA → Permanent Fix loop 26
Program hygiene 27
8. Deliverables Summary — M365 & Azure Incident Response & Escalation Runbook 27
Documentation 27
PowerShell scripts 28
Hero image 29
Appendix A — PowerShell Scripts 30

Executive Summary — M365 & Azure Incident Response & Escalation Runbook

The business problem

When a high-priority incident hits a Microsoft 365 or Azure estate — mail flow stops, users can't sign in, Teams calls fail, an Azure workload goes down, or an account is compromised — the difference between a 20-minute and a 4-hour outage is rarely the technology. It is whether the team has a **repeatable way to respond**: a clear severity call, named owners, calm client communication, a diagnostic path that reaches the real cause, a way to escalate to Microsoft, and a discipline that stops the same incident from happening again. Most teams improvise this under pressure, and the result is inconsistent response times, anxious clients, symptom-only fixes, and recurring outages.

What this package delivers

A complete, white-label **incident & problem management runbook** for Microsoft 365 and Azure — the operating system for high-priority and escalated incidents. It ships:

- A **severity & impact matrix** with response and update-cadence targets (P1–P4).
- The named **incident roles** — Incident Lead, Communications Lead, Scribe — and an on-call model.
- A **triage method** (impact × risk × effort) and a Service-Health-first discipline that separates a Microsoft outage from a tenant fault.
- **Per-workload diagnostic playbooks** with the real Microsoft console paths — Exchange Online mail flow, Entra ID sign-in/auth, Teams, SharePoint/OneDrive, Intune, and Azure resource health — plus Microsoft Service Health.
- **Client communication templates and a cadence** for the whole incident lifecycle.
- A **Microsoft escalation sheet** (with honest, plan-dependent severity guidance) and a **root-cause analysis** format.
- The **RCA → permanent fix loop** that converts each incident into a lasting hardening change.
- **Five PowerShell tools** for readiness validation, live discovery, safe remediation, responder-role setup, and MTTA/MTTR reporting.

The phase model

Detect → Triage → Communicate → Diagnose → Resolve → RCA → Permanent Fix — every incident, every time, with named artifacts and explicit gates.

Outcomes

- **Lower MTTA/MTTR** — consistent triage and diagnosis instead of improvisation; response no longer depends on who is on call.
- **Confident clients** — early, factual, on-cadence communication during the worst moments.

- **Real root causes** — diagnostics that reach the *why*, then escalate to Microsoft with evidence when the cause is platform-side.
- **Fewer repeat incidents** — the RCA-to-permanent-fix loop hardens the estate after each event.
- **Auditable managed service** — every incident has a named record, a severity, a timeline, and an RCA, demonstrating mature incident & problem management to clients and assessors.

Honest scope

This package is process, playbooks, and tooling — not a guarantee that every outage is locally fixable. When Microsoft is down, its value is correct triage, accurate client communication, and a well-formed escalation. Microsoft support severity and response are **plan-dependent**; the runbook tells you how to confirm your entitlement rather than promising an SLA you don't have. Response-time targets ship as sensible defaults and are overridden by your contractual SLAs.

Investment

A premium managed-services enablement package (~$2,495) that pays for itself the first time a P1 is resolved an hour faster — and again every time an RCA prevents the next one.

1. Solution Overview — M365 & Azure Incident Response & Escalation Runbook

What this solution is

This is a complete **operational runbook** for leading high-priority and escalated incidents across **Microsoft 365 and Azure**. It is an **incident & problem management package** for a managed-services team: it gives a responder everything needed to take a P1/P2 incident from first alert to a verified, permanent fix — the severity/impact matrix, the named roles, the triage method, the per-workload diagnostic playbooks, the client communication cadence and templates, the path to escalate to Microsoft support, the root-cause analysis (RCA) format, and the post-incident loop that converts a one-off fix into a permanent control.

It is a **playbook, not a wizard.** It pairs a phased incident lifecycle with named artifacts and per-workload diagnostic procedures so an on-call engineer, an incident lead, and a client-facing communicator can run a controlled, auditable response — every time, regardless of which engineer picks up the page.

The phase model

Every incident this runbook governs moves through seven phases, used verbatim throughout the package:

Detect → **Triage** → **Communicate** → **Diagnose** → **Resolve** → **RCA** → **Permanent Fix**

- **Detect** — the incident surfaces (monitoring alert, client report, Microsoft Service Health advisory, security signal).
- **Triage** — classify severity using the matrix, assign roles, and prioritize using impact × risk × effort.
- **Communicate** — open the client communication cadence with the right template at the right interval.

- **Diagnose** — run the named per-workload diagnostic playbook to reach a root cause (not just a symptom).
- **Resolve** — apply the fix (mitigation first, then full resolution), verify service is restored.
- **RCA** — produce the root-cause analysis record while the timeline is fresh.
- **Permanent Fix** — convert the RCA's corrective actions into a permanent control (a problem record), so the same incident does not recur.

This maps to the standard managed-services lifecycle of **Assess** → **Stabilize** → **Diagnose** → **Remediate** → **Verify** → **Review** → **Expand**: Triage is the Assess gate, Communicate+Resolve stabilize and remediate, RCA is the Review, and Permanent Fix is the Expand step that hardens the estate.

The named artifacts

The package stands up a small set of reusable artifacts, each named once and reused verbatim across the runbook, scripts, and client comms:

Artifact	Purpose
Severity & Impact Matrix – Incident – Triage	The single classification table mapping P1–P4 to impact, urgency, and response/cadence targets.
Incident Record Template – Incident – Lifecycle	The master record for one incident: timeline, severity, roles, actions, decisions, MTTA/MTTR.
Roles & On-Call Sheet – Incident – Response	Names the Incident Lead, Communications Lead, and Scribe per incident and the on-call rota.
Client Comms Templates – Incident – Communicate	Initial / Update / Resolution / Post-Incident message templates for clients and a status method.
Microsoft Escalation Sheet – Incident – Escalate	How and when to open and severity-set a Microsoft support case (plan-dependent).
RCA Template – Incident – Review	The structured root-cause analysis (timeline, root cause, contributing factors, corrective actions).
Problem Record – Incident – Permanent Fix	Tracks the permanent control derived from the RCA through to closure.
Diagnostic Playbooks – Incident – Diagnose	The per-workload procedures (Exchange Online, Entra ID, Teams, SharePoint/OneDrive, Intune, Azure).

The per-workload diagnostic playbooks

The Diagnose phase is driven by named, per-workload playbooks, each with the **real Microsoft console paths** an engineer follows:

1. **Exchange Online Mail Flow Playbook – Diagnose – Exchange** — message trace, transport rules, connectors, queues.
2. **Entra Sign-in & Auth Playbook – Diagnose – Identity** — sign-in logs, Conditional Access, MFA, risk.

3. **Teams Service Playbook – Diagnose – Teams** — call quality, meeting/chat health, Teams admin diagnostics.

4. **SharePoint & OneDrive Playbook – Diagnose – Collaboration** — site/sync health, sharing, storage.

5. **Intune Device & App Playbook – Diagnose – Endpoint** — enrollment, policy/app delivery, compliance.

6. **Azure Resource Health Playbook – Diagnose – Azure** — Resource Health, Service Health, Azure Monitor.

7. **Microsoft Service Health Playbook – Diagnose – Platform** — distinguish a Microsoft outage from a tenant issue.

How it works (the flow)

1. **Detect** the incident and open an **Incident Record Template – Incident – Lifecycle**.

2. **Triage**: score with the **Severity & Impact Matrix – Incident – Triage**; assign roles from the **Roles & On-Call Sheet – Incident – Response**.

3. **Check Microsoft Service Health first** — confirm whether this is a Microsoft-side outage before deep diagnosis.

4. **Communicate** to the client using the matching **Client Comms Templates – Incident – Communicate** at the cadence the severity demands.

5. **Diagnose** with the right **Diagnostic Playbook – Incident – Diagnose** until a root cause is identified.

6. **Escalate to Microsoft** via the **Microsoft Escalation Sheet – Incident – Escalate** if root cause is Microsoft-side or beyond local control.

7. **Resolve**: mitigate, then fully restore; verify; send the Resolution comms.

8. **RCA**: complete the **RCA Template – Incident – Review** within the agreed window.

9. **Permanent Fix**: open a **Problem Record – Incident – Permanent Fix** and drive the corrective control to closure.

Benefits

• **Faster, more consistent response** — every engineer runs the same triage, comms, and diagnostics, so MTTA/MTTR drop and don't depend on who is on call.

• **Clear client communication** — the templates and cadence remove the "what do we tell the client?" hesitation under pressure.

• **Root cause, not just symptom** — the playbooks push past restarting a thing toward *why* it broke.

• **No repeat incidents** — the RCA → Permanent Fix loop converts each incident into a hardening change.

• **Auditable** — every incident has a named record, a severity, a timeline, and an RCA — usable as managed-service evidence and for client trust.

Honest limitations

- **This is process and tooling, not a fix for every outage.** When the root cause is a Microsoft-side outage, the runbook's job is correct triage, accurate client comms, and escalation — not making Microsoft's service come back faster.
- **Microsoft support severity and response are plan-dependent.** What severity you may set, and the response commitment you receive, depend on your support plan (basic included support vs. Microsoft Unified / Premier). The runbook tells you how to check yours — it does not grant a faster SLA than your plan provides.
- **Response-time targets are yours to define.** The matrix ships with sensible defaults; your contractual SLAs override them.
- **Diagnostics require the right access.** Reaching root cause depends on having the reader/admin roles and log access in Requirements R1–R10; the runbook will stall at Diagnose without them.

The standard runbook model

This solution follows the **standard incident-management lifecycle** — a phased model with named artifacts, explicit gates, and a feedback loop:

Detect → Triage → Communicate → Diagnose → Resolve → RCA → Permanent Fix.

Each phase has an entry trigger, an exit criterion, and named deliverables. Triage gates whether the response escalates; RCA gates closure; and Permanent Fix gates whether the incident is allowed to recur. No incident is "closed" until its RCA is produced and any required Problem Record is opened.

2. Requirements — M365 & Azure Incident Response & Escalation Runbook

Each requirement is structured as **Description / Why / Verify / Implement-if-missing**. **Never assume a prerequisite exists** — verify each one before you rely on the runbook in a live incident. Requirements are numbered R1–R12. These layer on top of the shared `global-requirements-managed.md`.

> *Naming convention used throughout the runbook (reused verbatim in implementation): every created object is named `– –` , e.g.* `Responder Access Group – Identity – Response`*.*

R1. Responder identities with the right M365 / Azure roles (least privilege)

- **Description:** Each responder has a working account with the roles needed to **diagnose and act** across the in-scope workloads — at minimum reader-level visibility everywhere, with scoped admin where remediation is expected (Exchange Administrator, Teams Administrator, SharePoint Administrator, Intune Administrator, Azure Reader/Contributor, and a Security Reader).
- **Why:** The Diagnose phase stalls without log and console access; the Resolve phase stalls without scoped admin. Personal Global Admin for everyone is the wrong answer (blast radius, audit). Reader-first with scoped, PIM-eligible admin is the target.

• **Verify:** In **Entra admin center** (`https://entra.microsoft.com`) → **Roles & admins**, confirm each responder holds the expected role(s). In **Azure portal** (`https://portal.azure.com`) → **Subscriptions** → **Access control (IAM)**, confirm at least **Reader** on in-scope subscriptions.

• **Implement-if-missing:** Create the responder access group (R2) and assign the minimum roles; prefer **PIM eligible** activation over standing access. Run `scripts\INCIDENT-Set-ResponderRoles.ps1 -WhatIf` first, then apply.

R2. A defined responder group + on-call rota

• **Description:** A named Entra group `Responder Access Group – Identity – Response` containing the response team, and a documented on-call rota covering the hours your SLAs require.

• **Why:** Role assignment, alert routing, and "who is the Incident Lead right now?" all key off a known group and rota. An incident at 02:00 must have a named owner.

• **Verify:** In **Entra admin center** → **Groups**, confirm the group exists with current members. Confirm a published on-call schedule (PSA/calendar/paging tool) names the current on-call engineer.

• **Implement-if-missing:** Create the group (`INCIDENT-Set-ResponderRoles.ps1`), populate it, and stand up an on-call rota in your paging/PSA tool. Record both in the **Roles & On-Call Sheet – Incident – Response**.

R3. Access to Microsoft 365 Service Health

• **Description:** Responders can read **Service health** in the Microsoft 365 admin center and **Service Health** in the Azure portal.

• **Why:** The very first diagnostic question — "is this Microsoft or is this us?" — is answered here. Misclassifying a Microsoft outage as a tenant fault wastes the response.

• **Verify:** Open **Microsoft 365 admin center** (`https://admin.microsoft.com`) → **Health** → **Service health** and confirm it loads (requires Global Reader, Service Support Admin, or higher). Open **Azure portal** → **Service Health** and confirm advisories load.

• **Implement-if-missing:** Assign **Service Support Administrator** or **Global Reader** to responders; confirm the account can see **Health** → **Service health** and the Azure **Service Health** blade.

R4. Entra ID sign-in & audit log access

• **Description:** Responders can read **Sign-in logs** and **Audit logs** in Entra (and, where licensed, **Risky sign-ins / Risk detections**).

• **Why:** Identity is the most common root cause of "can't log in / can't get mail" incidents. Sign-in logs show failure reasons, Conditional Access results, and MFA outcomes; audit logs show what changed.

• **Verify: Entra admin center** → **Monitoring** → **Sign-in logs** and **Monitoring** → **Audit logs** both load. For risk data, **Protection** → **Identity Protection** requires **Entra ID P2**.

• **Implement-if-missing:** Assign a role that grants log read (**Reports Reader**, **Security Reader**, or **Global Reader**). For risk detections, confirm **Entra ID P2** licensing; if absent, document that risk-based diagnosis is unavailable.

R5. Exchange Online message trace + transport access

• **Description:** Responders can run **message trace** and read transport rules, connectors, and queues in the Exchange admin center, plus connect via the Exchange Online PowerShell module.

• **Why:** Mail-flow incidents (NDRs, delays, blocked senders, connector failures) are diagnosed through message trace and transport configuration — there is no other reliable path.

• **Verify: Exchange admin center** (`https://admin.exchange.microsoft.com`) → **Mail flow** → **Message trace** runs; → **Mail flow** → **Rules / Connectors** load. `Connect-ExchangeOnline` succeeds for the responder account.

• **Implement-if-missing:** Assign **Exchange Administrator** (or a custom RBAC role with View-Only Recipients + Message Tracking + Transport read). Install `ExchangeOnlineManagement` on the responder workstation.

R6. Defender / security signal access (for security-driven incidents)

• **Description:** Responders can read alerts and incidents in the **Microsoft Defender portal** (Defender for Office 365 / Endpoint / Identity, as licensed).

• **Why:** Many P1s start as a security signal (account compromise, mass mail, malware). The runbook needs to read these alerts to triage and to feed the Diagnose phase.

• **Verify: Microsoft Defender portal** (`https://security.microsoft.com`) → **Incidents & alerts** → **Incidents** loads for the responder account.

• **Implement-if-missing:** Assign **Security Reader** (read) or **Security Administrator** (act); confirm the relevant Defender workloads are licensed and onboarded. If not licensed, document the gap in the runbook.

R7. Azure Monitor / Resource Health access

• **Description:** Responders can read **Resource Health** and **Service Health** per resource and query **Azure Monitor** (Metrics, Log Analytics) for in-scope Azure workloads.

• **Why:** Azure resource incidents (VM down, App Service 5xx, SQL throttling) are diagnosed through Resource Health and Monitor metrics/logs.

• **Verify: Azure portal** → a resource → **Resource health** loads; **Monitor** → **Metrics** and any **Log Analytics workspace** are queryable by the responder (**Monitoring Reader** or **Reader**).

• **Implement-if-missing:** Assign **Monitoring Reader** (and **Reader**) on in-scope subscriptions/resources; confirm diagnostic settings send logs to a Log Analytics workspace so history exists at incident time.

R8. A Microsoft support plan (and knowledge of its severity rules)

• **Description:** A known Microsoft support entitlement for escalation — at minimum the support included with the subscription, ideally **Microsoft Unified** (the successor to Premier) — and documented knowledge of which case severities your plan permits and their response commitments.

• **Why:** When root cause is Microsoft-side or beyond local control, you escalate to Microsoft. **The severities you can set and the response you receive are plan-dependent** — included support, Microsoft Unified, and legacy Premier differ. You must know yours *before* the incident, not during it.

• **Verify: Microsoft 365 admin center** → **Support** → **Help & support** (or the **Microsoft Unified** portal `https://admin.microsoft.com` / `https://services.microsoft.com` for Unified customers) shows your entitlement and the severity options available when opening a case. Azure: **Azure portal** → **Help + support** → **Create a support request** shows the **severity** levels your plan allows (e.g. Sev A/B/C; Sev A typically requires a paid Azure support plan).

• **Implement-if-missing:** Confirm or purchase the appropriate support plan (Microsoft Unified for managed-service breadth, or an **Azure support plan** for Sev A response). Record entitlement details and severity rules in the **Microsoft Escalation Sheet – Incident – Escalate**.

R9. A ticketing / PSA system as the system of record

• **Description:** A ticketing or PSA platform where every incident is logged, timestamped, and tracked through closure.

• **Why:** The **Incident Record Template – Incident – Lifecycle**, MTTA/MTTR metrics, and the audit trail all need a durable home. Email and chat are not a system of record.

• **Verify:** Confirm a ticket type/queue exists for incidents and that responders can create and update tickets with timestamps.

• **Implement-if-missing:** Configure an incident ticket type in your PSA carrying the fields the Incident Record Template needs (severity, detected/acknowledged/resolved times, roles, RCA link). Map the template to those fields.

R10. Defined on-call communication channels + a client status method

• **Description:** A real-time responder channel (e.g. a dedicated chat/bridge for war-rooming) **and** a defined method to communicate status to clients — a status page, an email distribution, or a documented call/contact path per client.

• **Why:** The Communicate phase requires both an internal coordination channel and an outward client channel. Improvising the client channel mid-incident erodes trust.

• **Verify:** Confirm a standing incident bridge/channel exists and that each managed client has a documented status/comms method and an after-hours contact.

• **Implement-if-missing:** Create the incident war-room channel; for each client, record the comms method and emergency contact in the **Client Comms Templates – Incident – Communicate** front matter.

R11. Defined response-time / SLA targets

• **Description:** Documented response and update-cadence targets per severity — either your contractual SLAs or the matrix defaults adopted as policy.

• **Why:** Triage and the comms cadence are meaningless without targets; "P1 = update every 30 minutes" must be a stated commitment, not an ad-hoc decision.

• **Verify:** Confirm the **Severity & Impact Matrix – Incident – Triage** carries response and cadence targets and that these match (or exceed) any client SLAs.

• **Implement-if-missing:** Populate the matrix targets from contracts; where no contract specifies, adopt the defaults in §03 as internal policy.

R12. Change-control & rollback posture for remediations

• **Description:** A lightweight change path for incident remediations, including approval for high-risk fixes and a back-out for each, plus the discipline to record changes in the incident record.

• **Why:** Incident fixes are applied under pressure; an un-recorded or irreversible change can turn one incident into two. Emergency change still needs a record and a back-out.

• **Verify:** Confirm an **emergency change** path exists and that responders know when a fix needs approval vs. when to act-then-document. Confirm `INCIDENT-Remediation.ps1` defaults to `-WhatIf`.

• **Implement-if-missing:** Define an emergency-change rule (what may be done without prior approval, what may not), and require every remediation to be logged in the **Incident Record Template – Incident – Lifecycle**. See also `global-requirements-managed.md`.

3. Implementation — M365 & Azure Incident Response & Escalation Runbook

This is the operational build-out **and** the live runbook. Part A stands up the named artifacts once (the matrix, templates, playbooks, escalation sheet, RCA, and responder access). Part B is the per-incident runbook you execute every time, phase by phase, with full Microsoft console navigation paths.

Steps are imperative and ordered. Every created object uses the naming standard – – verbatim. **Never assume a prerequisite exists** — each step that depends on access points back to a Requirement (R1–R12) to verify and implement-if-missing.

Part A — Stand up the runbook (one-time)

A1. Build the Severity & Impact Matrix – Incident – Triage

1. Create the artifact **Severity & Impact Matrix – Incident – Triage** in your PSA/knowledge base (R9). It is a table mapping **severity (P1–P4)** against **business impact**, **urgency**, **response-time target**, and **update cadence**.

2. Populate it from your client SLAs (R11). Where no SLA specifies, adopt these defaults:

Severity	Definition (impact)	Examples	Response target	Update cadence
P1 — Critical	Total or near-total outage of a critical service for many/all users; no workaround.	Mail flow stopped tenant-wide; all users can't sign in; suspected active account compromise.	Immediate (≤15 min)	Every 30 min
P2 — High	Major degradation or outage for a department/site, or a critical service degraded with a partial workaround.	One site can't access SharePoint; Teams calls failing for a region; recurring sign-in failures for a group.	≤30 min	Every 60 min
P3 — Moderate	Single user or small group impacted; workaround exists; not business-critical.	One mailbox delayed; one device won't enroll.	≤4 business hours	Daily
P4 — Low	Minor issue, question, or cosmetic; no business impact.	Single non-urgent config query.	≤1 business day	On change

1. **Verify:** confirm the matrix carries response and cadence targets and that they match or exceed every client SLA (R11). Store one canonical copy; reference it from the Incident Record Template.

Note (severity ≠ Microsoft severity): *these P1–P4 levels are *your* internal/client severities. Microsoft support uses its own severity scheme (Azure Sev A/B/C; M365 Critical/Severe/etc.), and which you may set is plan-dependent — see A5 and Part B Phase 6.*

A2. Build the Incident Record Template – Incident – Lifecycle

1. In your PSA (R9), create the incident ticket type **Incident Record Template – Incident – Lifecycle** with these fields:

• Incident ID, Title, Client/Tenant, **Severity** (from A1).

• **Timestamps:** Detected, Acknowledged (drives MTTA), Mitigated, Resolved (drives MTTR), Closed.

• **Roles:** Incident Lead, Communications Lead, Scribe (from A3).

• **Timeline / running log** (every action and decision, timestamped — the Scribe owns this).

• **Affected workload(s)** and the Diagnostic Playbook used.

• **Microsoft case number** (if escalated) and Microsoft severity.

• **Root cause** (link to RCA), **Corrective actions**, **Problem Record** link.

1. **Verify:** create a test incident and confirm all fields capture and timestamp correctly.

A3. Build the Roles & On-Call Sheet – Incident – Response

1. Create **Roles & On-Call Sheet – Incident – Response** documenting the three per-incident roles and the on-call rota (R2):

• **Incident Lead** — owns the response, makes decisions, sets severity, decides escalation. The single throat to choke.

• **Communications Lead** — owns all client and stakeholder comms (A4); shields the Lead and engineers from status requests.

• **Scribe** — maintains the timeline in the Incident Record; captures who-did-what-when for the RCA.

1. For a small team, one person may hold multiple roles — but the **Incident Lead role is always named** before diagnosis begins.

2. Link the on-call rota (R2) so "current Incident Lead" is always resolvable.

3. **Verify:** confirm the current on-call engineer is named and reachable through the paging tool.

A4. Build the Client Comms Templates – Incident – Communicate

1. Create **Client Comms Templates – Incident – Communicate** with four templates and a defined client status method (R10):

Initial notification (sent at first triage):

**Subject: [P{n}] Service incident affecting {service} — investigating* We are aware of an issue affecting {service} that began at approximately {time}. Impact: {who/what}. Our team is investigating and treating this as Priority {n}. Next update by {time + cadence}.*

Update (sent every cadence interval):

**Subject: [P{n}] {service} incident — update {hh:mm}* Status: {investigating / mitigated / monitoring}. What we know: {brief}. What we're doing: {brief}. Workaround (if any): {…}. Next update by {time}.*

Resolution:

**Subject: [P{n}] {service} incident — RESOLVED* Service was restored at {time}. Cause (preliminary): {brief}. We are completing a full root-cause analysis and will share findings by {date}. Thank you for your patience.*

Post-incident summary (sent after RCA):

**Subject: Post-incident report — {service} incident {date}* Summary, timeline, root cause, corrective actions, and the permanent fix we are implementing. (Body = the client-facing extract of the RCA Template.)*

1. Record each client's **status method** (status page / email DL / call path) and after-hours contact (R10) in the template front matter.

2. **Verify:** send a test Initial+Update to an internal address; confirm the cadence reminder fires.

A5. Build the Microsoft Escalation Sheet – Incident – Escalate

1. Create **Microsoft Escalation Sheet – Incident – Escalate** recording, for **each tenant/subscription**:

• The Microsoft **support entitlement** (included support / **Microsoft Unified** / legacy Premier / **Azure support plan**) — confirm at **Microsoft 365 admin center** → **Support** → **Help & support**, the **Microsoft Unified** portal (`https://services.microsoft.com`) for Unified customers, and **Azure portal** → **Help + support** (R8).

• The **case severities your plan permits** and their response commitments. *Be explicit that this is plan-dependent:* Azure **Sev A** (critical, fastest response) typically requires a **paid Azure support plan**; M365 case severity options likewise vary by entitlement. Do not promise a severity your plan can't set.

• **How to open a case:**

• **M365:** `https://admin.microsoft.com` → **Support** → **Help & support** → **Contact support** (or "New service request"); attach the message trace / sign-in log / correlation IDs you gathered in Diagnose.

• **Azure:** `https://portal.azure.com` → **Help + support** → **Create a support request**; pick the resource, **Issue type**, and the **severity** your plan allows.

• **Unified:** `https://services.microsoft.com` → submit/manage requests against the Unified contract.

• The **internal escalation chain** (when the Incident Lead escalates to a senior engineer / management) and the trigger conditions.

1. **Verify:** confirm each tenant's entitlement and the severity list shown when you begin (do not submit) a case; record screenshots/notes.

A6. Build the Diagnostic Playbooks – Incident – Diagnose

Author the seven named per-workload playbooks below — each is a short, ordered procedure with the **exact Microsoft console path**. These are referenced verbatim in Part B Phase 4.

*A6.1 Microsoft Service Health Playbook – Diagnose – Platform *(run this FIRST)**

1. **M365:** `https://admin.microsoft.com` → **Health** → **Service health**. Review active issues/advisories for the affected workload; note the **issue ID** and status.

2. **Azure:** `https://portal.azure.com` → **Service Health** → **Service issues** (and **Health advisories**, **Security advisories**, **Planned maintenance**). Filter to your subscriptions/regions.

3. If an active Microsoft issue matches the symptom, this is a **Microsoft-side incident** — switch to "track + communicate + escalate" (Part B Phases 3 & 6), not deep local diagnosis.

A6.2 Exchange Online Mail Flow Playbook – Diagnose – Exchange

1. `https://admin.exchange.microsoft.com` → **Mail flow** → **Message trace**. Run a trace for the affected sender/recipient and time window; read the delivery status / failure reason.

2. **Mail flow** → **Rules** — check for a transport rule blocking/redirecting the message.

3. **Mail flow** → **Connectors** — check inbound/outbound connector health (relevant to hybrid/third-party gateways).

4. **Defender:** `https://security.microsoft.com` → **Email & collaboration** → **Threat Explorer / Quarantine** — confirm the message wasn't quarantined or ZAP'd.

5. For deeper data: `Connect-ExchangeOnline` (R5) then `Get-MessageTrace` / `Get-MessageTraceDetail`.

A6.3 Entra Sign-in & Auth Playbook – Diagnose – Identity

1. `https://entra.microsoft.com` → **Monitoring** → **Sign-in logs**. Filter to the affected user/time; open a failed sign-in and read **Failure reason**, **Conditional Access** result, and **MFA** detail.

2. **Protection** → **Conditional Access** → **Policies** — check whether a CA policy is blocking; use **What If** to simulate.

3. **Protection** → **Identity Protection** → **Risky sign-ins / Risk detections** (requires **Entra ID P2**, R4) — check for risk-driven blocks or compromise.

4. **Monitoring** → **Audit logs** — see recent changes (policy, user, app) that could have caused the regression.

A6.4 Teams Service Playbook – Diagnose – Teams

1. `https://admin.teams.microsoft.com` → **Users** → select the user → **Meetings/Call history**; for quality, open a specific call.

2. **Analytics & reports** → **Call quality dashboard (CQD)** — assess call/meeting quality trends.

3. Confirm underlying dependencies: Teams relies on Exchange (calendar) and SharePoint (files) — cross-check A6.2 and A6.5 if those are implicated.

4. Check **Service health** (A6.1) — Teams incidents are frequently platform-side.

A6.5 SharePoint & OneDrive Playbook – Diagnose – Collaboration

1. `https://admin.microsoft.com` → **(SharePoint admin center)** `https://-admin.sharepoint.com` → **Sites** → **Active sites** — check the affected site's status and storage.

2. For sync issues: on the device, OneDrive sync health and the **Sync admin reports** in the SharePoint admin center.

3. **Sharing:** site/tenant external-sharing settings if access-denied is the symptom.

4. Check **Service health** (A6.1) for SharePoint/OneDrive advisories.

A6.6 Intune Device & App Playbook – Diagnose – Endpoint

1. `https://intune.microsoft.com` → **Devices** → **All devices** → select the device → **Device compliance** and **Managed Apps**; review last check-in.

2. **Devices** → **Monitor** → **Enrollment failures** for enrollment incidents.

3. **Apps** → **Monitor** → **App install status** for delivery failures.

4. **Reports** → **Endpoint analytics** for fleet-wide signals.

A6.7 Azure Resource Health Playbook – Diagnose – Azure

1. `https://portal.azure.com` → the affected resource → **Resource health** — read current/historical health and Microsoft's stated cause (platform vs. customer-initiated).

2. **Monitor** → **Metrics** — chart the resource's key metrics around the incident time; **Monitor** → **Logs** (Log Analytics) for diagnostic logs (R7).

3. **Service Health** (A6.1) — confirm whether a regional Azure issue is in play.

4. For activity/changes: the resource's **Activity log** — see who changed what.

Verify (A6): walk each playbook against a known-good resource and confirm every console path resolves with the responder's access (R3–R7).

A7. Build the RCA Template – Incident – Review and the Problem Record – Incident – Permanent Fix

1. Create **RCA Template – Incident – Review** with sections: **Summary**, **Impact** (who/what/duration), **Timeline** (from the Incident Record), **Root cause** (the *why*, via 5-Whys), **Contributing factors**, **What went well / what didn't**, **Corrective actions** (each with owner + due date), and **Detection/Prevention improvements**.

2. Create **Problem Record – Incident – Permanent Fix** to track each corrective action that is a *permanent control* (not a one-off fix) through to closure, linked back to the RCA and the originating incident.

3. **Verify:** confirm both templates link bidirectionally to the Incident Record (A2).

A8. Provision responder access (R1, R2)

1. Run `scripts\INCIDENT-Set-ResponderRoles.ps1 -WhatIf` to preview creation of `Responder Access Group – Identity – Response` and the reader/scoped-admin role assignments.

2. Review the preview; then re-run with `-Apply` to create the group and assign roles. Prefer **PIM-eligible** for admin roles.

3. Run `scripts\INCIDENT-Validation.ps1` to confirm responder readiness: roles present, log access, support plan — expect **PASS** on all checks (R1–R8).

4. **Verify:** sign in as a responder and confirm each console path in A6 loads.

Part B — The per-incident runbook (every incident)

Phase 1 — Detect

1. The incident surfaces via a monitoring alert, a client report, a Microsoft Service Health advisory, or a security signal (Defender).

2. **Acknowledge** immediately and open an **Incident Record Template – Incident – Lifecycle** (A2). Record **Detected** and **Acknowledged** timestamps (these drive MTTA).

3. Capture the raw symptom verbatim and the reporter/source.

Phase 2 — Triage

1. The on-call engineer assumes (or assigns) the **Incident Lead** role and, for P1/P2, names a **Communications Lead** and **Scribe** from the **Roles & On-Call Sheet – Incident – Response** (A3).

2. Classify severity with the **Severity & Impact Matrix – Incident – Triage** (A1) using **impact × urgency**. To prioritize among competing actions, weigh **impact × risk × effort** — do the highest-impact, lowest-risk, lowest-effort mitigation first.

3. **Run the Microsoft Service Health Playbook – Diagnose – Platform (A6.1) now** — answer "is this Microsoft or us?" before committing to deep diagnosis.

4. Record severity and roles in the Incident Record. This is the gate: P1/P2 trigger the full response; P3/P4 follow the lighter cadence.

Phase 3 — Communicate

1. The **Communications Lead** sends the **Initial notification** from **Client Comms Templates – Incident – Communicate** (A4) via the client's status method, at the severity's response target (R11).

2. Set a recurring reminder at the matrix **update cadence** and send the **Update** template each interval — even "no change yet" is an update.

3. Keep internal coordination in the incident war-room channel (R10); keep client comms calm, factual, and free of internal jargon.

Phase 4 — Diagnose

1. Select the **Diagnostic Playbook – Incident – Diagnose** matching the affected workload (A6.2–A6.7) and follow its ordered console steps.

2. Drive toward **root cause**, not symptom — capture correlation IDs, message-trace IDs, sign-in error codes, and Resource Health statements; the Scribe logs each finding with a timestamp.

3. If multiple workloads are implicated, run the dependent playbooks (e.g. Teams → Exchange/SharePoint).

4. If the playbook points at a Microsoft-side cause, proceed to Phase 6 (escalate) in parallel with continued communication.

5. (Optional) Run `scripts\INCIDENT-Discovery.ps1` to snapshot current Service Health, recent high-severity sign-ins/risk, and Azure resource health in one pass to seed diagnosis.

Phase 5 — Resolve

1. **Mitigate first** (restore service / workaround), **then** fix fully — record both in the Incident Record with timestamps (**Mitigated**, then **Resolved**, which drives MTTR).

2. For common, well-understood fixes, use `scripts\INCIDENT-Remediation.ps1` — it defaults to `-WhatIf`; review the preview, then `-Apply` (e.g. revoke a compromised user's sessions; re-enable a disabled mail-flow connector for a *named* object). High-risk fixes follow the emergency-change rule

(R12).

3. **Verify the fix** against the original symptom (re-run the relevant playbook step; confirm with the reporter).

4. The Communications Lead sends the **Resolution** template (A4).

Phase 6 — Escalate to Microsoft (when needed)

1. Escalate when root cause is **Microsoft-side** (confirmed in Service Health) or **beyond local control**, using the **Microsoft Escalation Sheet – Incident – Escalate** (A5).

2. Open the case at the right portal (M365 / Azure / Unified) and set the **severity your plan permits** — *remember severity options and response are plan-dependent* (R8). Attach the diagnostic evidence gathered in Phase 4.

3. Record the **Microsoft case number** and severity in the Incident Record; keep the client updated that an escalation is in progress.

Phase 7 — RCA → Permanent Fix

1. Within the agreed window (e.g. ≤5 business days for P1), complete the **RCA Template – Incident – Review** (A7) from the Incident Record timeline; use **5-Whys** to reach the true root cause.

2. Send the **Post-incident summary** (A4) — the client-facing extract of the RCA.

3. For every corrective action that is a *permanent control*, open a **Problem Record – Incident – Permanent Fix** (A7) with an owner and due date, and drive it to closure (a config hardening, a new monitor/alert, a policy change, a documentation/runbook update).

4. Run `scripts\INCIDENT-Report.ps1` to compute **MTTA/MTTR** and append this incident to the trend metrics.

5. **Close** the incident only when the RCA exists and any required Problem Record is open. The loop is complete when the permanent fix lands — converting this incident into estate hardening (the Expand step).

5. Testing — M365 & Azure Incident Response & Escalation Runbook

The runbook is validated with a **tabletop / simulated incident** (a controlled "game day"), not by waiting for a real P1. Each test below has an explicit **PASS/FAIL gate**. The overall acceptance gate is: **a simulated incident is triaged correctly, comms are sent on cadence, diagnosis reaches a real root cause, an RCA is produced, and a permanent fix is logged.** Run the full sequence at least once before relying on the runbook, and re-run after any major change to the estate or team.

> *Run readiness checks (T1–T2) with* `scripts\INCIDENT-Validation.ps1` *and capture the simulation evidence in a real* ***Incident Record Template – Incident – Lifecycle*** *flagged* `SIMULATION`.

T1. Responder readiness — roles & access *(gate)*

• **Action:** Run `scripts\INCIDENT-Validation.ps1`. Independently spot-check that a responder can open: **M365 admin** → **Health** → **Service health**, **Entra** → **Monitoring** → **Sign-in logs**, **Exchange admin** → **Mail flow** → **Message trace**, **Azure** → **Service Health** and a resource's **Resource health**, and **Defender** → **Incidents** (R3–R7).

• **PASS:** Validation returns **PASS** on roles, log access, and the consoles all load for the responder account.

• **FAIL:** Any console denies access or Validation reports a missing role → fix via the cited Requirement before continuing.

T2. Support-plan & escalation readiness *(gate)*

• **Action:** From the **Microsoft Escalation Sheet – Incident – Escalate**, confirm the recorded Microsoft entitlement matches what the portals show. Begin (do **not** submit) a support request in **Azure** → **Help + support** → **Create a support request** and in **M365** → **Support** → **Help & support**; observe the **severity** options offered.

• **PASS:** Entitlement is documented and the severity options the runbook claims are the ones actually offered (plan-dependent reality matches the sheet).

• **FAIL:** The sheet over-states available severity or the entitlement is unknown → correct the sheet / acquire the plan (R8).

T3. Triage correctness *(gate)*

• **Action:** Present 3 scripted scenarios (e.g. "all users can't sign in"; "one mailbox delayed"; "Teams calls failing for one site"). The responder classifies each with the **Severity & Impact Matrix – Incident – Triage** and names the **Incident Lead** (and, for P1/P2, Comms Lead + Scribe).

• **PASS:** All three are classified to the expected severity with correct response/cadence targets, and roles are named before diagnosis.

• **FAIL:** A scenario is mis-prioritized or no Incident Lead is named → re-train on A1/A3; refine the matrix wording.

T4. Service Health discrimination *(gate)*

• **Action:** In the simulation, require the responder to run the **Microsoft Service Health Playbook – Diagnose – Platform** *before* deep diagnosis and state whether the cause is Microsoft-side or tenant-side.

• **PASS:** The responder checks both **M365 Health** → **Service health** and **Azure** → **Service Health** and correctly states is-this-Microsoft-or-us before diving in.

• **FAIL:** The responder skips Service Health and begins local diagnosis → enforce A6.1-first in Phase 2.

T5. Communication on cadence *(gate)*

• **Action:** During the simulation, the Communications Lead sends the **Initial** notification within the severity's response target and an **Update** at each cadence interval, using **Client Comms Templates – Incident – Communicate** (to an internal test recipient).

• **PASS:** Initial sent within target; at least two cadence Updates sent on time; tone is factual and jargon-free.

• **FAIL:** Late/missing comms or improvised wording → fix the cadence reminder and template usage (A4, R11).

T6. Diagnosis reaches root cause *(gate)*

• **Action:** Seed a diagnosable fault scenario (or replay a past incident). The responder follows the matching **Diagnostic Playbook – Incident – Diagnose** and produces a stated **root cause** with supporting evidence (trace ID, sign-in error code, Resource Health statement).

• **PASS:** A genuine root cause (the *why*) is reached with cited evidence — not merely "restarted it and it works."

• **FAIL:** Diagnosis stops at the symptom → review the relevant A6 playbook; confirm log access (R4–R7).

T7. Remediation safety *(gate)*

• **Action:** Run `scripts\INCIDENT-Remediation.ps1` against a named non-production object **without** `-Apply`; then with `-Apply` in a sanctioned test.

• **PASS:** Default run makes **no change** and prints the intended action (`-WhatIf`); `-Apply` performs only the named, scoped action and it is logged in the Incident Record (R12).

• **FAIL:** The script changes anything without `-Apply`, or acts beyond the named object → stop; correct before live use.

T8. RCA produced *(gate)*

• **Action:** After the simulated resolution, complete the **RCA Template – Incident – Review** from the Incident Record timeline, including a 5-Whys root cause and corrective actions with owners/dates.

• **PASS:** A complete RCA exists within the agreed window with a real root cause and owned corrective actions.

• **FAIL:** RCA missing, late, or root-cause-free → enforce Phase 7; refine the template (A7).

T9. Permanent fix logged *(gate — overall)*

• **Action:** For at least one corrective action, open a **Problem Record – Incident – Permanent Fix** with an owner and due date and link it to the RCA and incident. Run `scripts\INCIDENT-Report.ps1` and confirm MTTA/MTTR computed.

• **PASS:** A Problem Record exists for the permanent control, the loop is closed (incident → RCA → permanent fix), and the metrics report generates.

• **FAIL:** No permanent fix tracked (the incident can recur) or metrics don't compute → the overall gate fails; do not certify the runbook until closed.

Acceptance gate (overall)

The runbook **passes** when, in a single end-to-end simulation, **all of T1–T9 PASS**: readiness confirmed, triage correct, Service Health checked, comms on cadence, diagnosis reaching root cause, safe remediation, a produced RCA, and a logged permanent fix — with MTTA/MTTR captured. Record the simulation Incident Record and the Validation/Report CSVs as the certification evidence.

6. Troubleshooting — M365 & Azure Incident Response & Escalation Runbook

Failure modes you hit while *running* the runbook (not the incidents themselves), grouped by phase, each with a likely cause and a fix. For the incidents themselves, use the per-workload **Diagnostic Playbooks – Incident – Diagnose** in §03 (A6).

Access & visibility (Detect / Diagnose)

Symptom: a responder can't open Service health / sign-in logs / message trace.

- *Cause:* missing reader/admin role (R3–R7). *Fix:* assign **Global Reader** / **Service Support Administrator** (Service health), **Reports/Security Reader** (sign-in logs), **Exchange Administrator** or custom RBAC (message trace), **Monitoring Reader** (Azure). Re-run `INCIDENT-Validation.ps1`.

Symptom: `Connect-ExchangeOnline` **or** `Connect-MgGraph` **fails for a responder.**

- *Cause:* module not installed, or the account lacks consent/role. *Fix:* `Install-Module ExchangeOnlineManagement, Microsoft.Graph -Scope CurrentUser`; confirm the role (R5/R4); for Graph, ensure the requested scopes are consented.

Symptom: Identity Protection / Risky sign-ins blade is empty or hidden.

- *Cause:* **Entra ID P2** not licensed (R4). *Fix:* license P2, or document that risk-based diagnosis is unavailable and rely on raw sign-in logs.

Symptom: Azure Monitor has no historical data at incident time.

- *Cause:* diagnostic settings weren't sending logs to Log Analytics before the incident (R7/G4). *Fix:* enable diagnostic settings now with adequate retention; you cannot retroactively create history.

Service Health discrimination (Triage)

Symptom: hours spent diagnosing a tenant fault that was actually a Microsoft outage.

- *Cause:* skipped the **Microsoft Service Health Playbook – Diagnose – Platform** (A6.1). *Fix:* enforce Service-Health-first in Phase 2; check **both** `admin.microsoft.com → Health → Service health` **and** `portal.azure.com → Service Health`.

Symptom: Service health shows nothing but users are clearly impacted.

- *Cause:* localized/region issue not yet posted, or a tenant-specific cause. *Fix:* don't wait for Microsoft to post — proceed to the workload playbook; capture correlation IDs to escalate if it proves Microsoft-side.

Triage & roles

Symptom: incidents get mis-prioritized or response is inconsistent.

- *Cause:* matrix targets not aligned to SLAs, or no Incident Lead named (R11/A3). *Fix:* sync the **Severity & Impact Matrix – Incident – Triage** to contracts; require an Incident Lead before diagnosis.

Symptom: engineers swamped with status requests, can't focus on the fix.

- *Cause:* no Communications Lead routing comms. *Fix:* for P1/P2, always name a Communications Lead (A3); route all status questions through them.

Communication

Symptom: client complains they were "left in the dark."

- *Cause:* missed the cadence or never sent the Initial notification. *Fix:* set the cadence reminder at triage; send the Initial even with no answers yet; send "no change" updates on schedule (A4, R11).

Symptom: client confused or alarmed by an update.

- *Cause:* internal jargon or premature speculation on cause. *Fix:* use the templates verbatim; state impact, action, workaround, and next-update time only; keep cause out until evidenced.

Escalation to Microsoft

Symptom: can't set the severity you expected when opening a case.

- *Cause:* the severity is **plan-dependent** (R8) — e.g. Azure **Sev A** needs a paid Azure support plan. *Fix:* correct the **Microsoft Escalation Sheet – Incident – Escalate** to reflect what your plan actually permits; if you need faster response, acquire **Microsoft Unified** / an Azure support plan.

Symptom: Microsoft case stalls or bounces back for "more info."

- *Cause:* opened with symptoms only. *Fix:* attach the Phase-4 evidence (correlation IDs, message traces, sign-in error codes, Resource Health statements); reference the affected resource/region precisely.

Remediation

Symptom: `INCIDENT-Remediation.ps1` didn't change anything.

- *Cause:* it defaults to `-WhatIf` (by design). *Fix:* review the preview, then re-run with `-Apply`. Never add `-Apply` blindly.

Symptom: a remediation made things worse / hit the wrong object.

- *Cause:* ran broadly instead of on a named object, or skipped the emergency-change rule (R12). *Fix:* always target a named object; follow the back-out in the Incident Record; record the change.

RCA → Permanent Fix loop

Symptom: the same incident keeps recurring.

- *Cause:* fixes stopped at mitigation; no permanent fix tracked. *Fix:* enforce Phase 7 — complete the **RCA Template – Incident – Review** with 5-Whys and open a **Problem Record – Incident – Permanent Fix** with an owner and date.

Symptom: RCA is late, thin, or missing.

- *Cause:* no contemporaneous timeline (Scribe role skipped) or no agreed window. *Fix:* keep the Scribe's timeline live during the incident; set an RCA SLA (e.g. ≤5 business days for P1) and gate closure on the RCA existing.

Symptom: MTTA/MTTR can't be computed.

- *Cause:* missing Detected/Acknowledged/Resolved timestamps in the Incident Record. *Fix:* make those fields mandatory in the **Incident Record Template – Incident – Lifecycle** (A2); re-run `INCIDENT-Report.ps1`.

7. Best Practices — M365 & Azure Incident Response & Escalation Runbook

These are the operating disciplines that make the runbook effective in practice. They assume the named artifacts from §03 are in place and reused verbatim.

Triage & prioritization

- **Service Health first, always.** Run the **Microsoft Service Health Playbook – Diagnose – Platform** before any deep local diagnosis. The cheapest minute in any incident is the one that tells you it's a Microsoft outage.
- **Prioritize by impact × risk × effort.** Take the highest-impact, lowest-risk, lowest-effort mitigation first. A 30-second workaround that restores 200 users beats a 2-hour "proper" fix while they wait.
- **Mitigate before you perfect.** Restore service first; do the elegant root-cause fix after. Record the mitigation and the resolution as separate timeline entries (they drive different metrics).
- **Sever the symptom from the cause.** "It works again" after a restart is a mitigation, not a root cause. Phase 7 is non-negotiable for P1/P2.

Roles & coordination

- **Name the Incident Lead before diagnosis.** Even a one-person response names the Lead in the record. Diffuse ownership is how incidents drift.
- **Protect the engineers.** For P1/P2, route all client/stakeholder status requests through the **Communications Lead** so the people fixing the problem can fix the problem.
- **The Scribe is not optional under pressure.** A contemporaneous, timestamped timeline is what makes the RCA accurate and fast. Reconstructing it from memory afterward loses the truth.

Client communication

- **Communicate early, even with little to say.** The Initial notification builds trust precisely because it goes out before you have answers. Silence is what erodes confidence.
- **Hold the cadence religiously.** Send the Update at every interval, including "still investigating, next update by HH:MM." Missing a promised update is its own incident.

• **Calm, factual, jargon-free.** Clients want impact, what you're doing, any workaround, and the next update time — not packet captures.

• **Never speculate publicly on cause** until you have evidence; preliminary cause goes in Resolution, confirmed cause goes in the Post-Incident summary.

Diagnosis

• **Capture correlation IDs as you go** (message-trace IDs, sign-in request IDs, Resource Health correlation IDs). They are gold for a Microsoft escalation and worthless if you have to re-derive them.

• **Follow the named playbook in order.** The ordering encodes the fastest path to root cause for that workload; ad-hoc clicking is slower and skips checks.

• **Mind the dependency chain.** Teams leans on Exchange (calendar) and SharePoint (files); SharePoint/OneDrive lean on identity. Diagnose the dependency when the surface symptom doesn't add up.

Escalation to Microsoft

• **Know your plan before the incident.** Re-confirm the **Microsoft Escalation Sheet – Incident – Escalate** entitlement periodically — *severity options and response are plan-dependent* and entitlements change. Azure **Sev A** generally needs a paid Azure support plan; **Microsoft Unified** gives managed-service breadth.

• **Escalate with evidence, not just symptoms.** Attach the diagnostic artifacts from Phase 4; a case with correlation IDs and message traces moves far faster.

• **Escalate internally on a clock, too.** Define when the Incident Lead pulls in a senior engineer or management — don't let a stuck P1 stay quietly stuck.

Remediation safety

• `-WhatIf` **by default,** `-Apply` **deliberately.** `INCIDENT-Remediation.ps1` previews by default for a reason. Read the preview; apply only the named, scoped action.

• **Act on named objects only.** Revoke *this* user's sessions; re-enable *this* connector. Never run a remediation broadly under incident pressure.

• **Emergency change still has a record and a back-out.** Define what may be done before approval and what may not (R12); log every change in the Incident Record.

RCA → Permanent Fix loop

• **5-Whys to the real cause.** Stop at "the certificate expired" and you'll be back; ask why it expired un-noticed and you'll add the monitor that prevents recurrence.

• **Every RCA action has an owner and a date.** Actions without owners don't happen.

• **Convert fixes into controls.** A permanent fix is usually a new alert/monitor, a config hardening, a policy, or a runbook update — track it as a **Problem Record – Incident – Permanent Fix** to closure. This is the Expand step that compounds reliability.

• **Feed the metrics.** Run `INCIDENT-Report.ps1` after each incident; a falling MTTA/MTTR and falling repeat-incident count is the proof the program works.

Program hygiene

• **Run a tabletop quarterly.** The §05 simulation keeps the team fluent and surfaces access gaps before a real P1 does.

• **Keep access least-privilege and PIM-eligible.** Standing Global Admin for the whole team is an audit finding and a breach blast-radius; reader-first with eligible scoped admin is the target.

• **Keep the matrix and templates in sync with contracts.** When a client SLA changes, update the **Severity & Impact Matrix – Incident – Triage** the same day.

8. Deliverables Summary — M365 & Azure Incident Response & Escalation Runbook

This package is a complete, enterprise-grade incident & problem management kit for leading high-priority and escalated incidents across Microsoft 365 and Azure. It contains the documentation runbook, five PowerShell tools, and a hero image.

Documentation

File	What it covers
`01-solution-overview.md`	What the runbook is, the **Detect** → **Triage** → **Communicate** → **Diagnose** → **Resolve** → **RCA** → **Permanent Fix** phase model, the named artifacts, the per-workload diagnostic playbooks, how it works, benefits, and honest limitations.
`02-requirements.md`	R1–R12 prerequisites (Description / Why / Verify / Implement-if-missing): responder roles + reader access, Service Health access, Entra/Exchange/Azure log access, a Microsoft support plan, a PSA, comms channels, SLAs, and change control.
`global-requirements-managed.md`	Baseline requirements common to every Managed-category engagement (tenant/subscriptions, roles, licensing, monitoring + system of record, change control, backup/rollback).
`03-implementation.md`	Part A stands up the matrix, templates, playbooks, escalation sheet, RCA, and responder access; Part B is the per-incident runbook with full Microsoft console navigation paths for diagnosis and escalation.

File	What it covers
`05-testing.md`	Numbered tests (T1–T9) with PASS/FAIL gates, validated through a tabletop/simulated incident: triage correct, comms on cadence, diagnosis reaching root cause, RCA produced, permanent fix logged.
`06-troubleshooting.md`	Common failure modes across access/visibility, Service Health discrimination, per-workload diagnosis, escalation, remediation, comms, and the RCA loop — with fixes.
`07-best-practices.md`	The operating disciplines: Service-Health-first triage, role coordination, client comms, evidence-led diagnosis and escalation, remediation safety, and the RCA→permanent-fix loop.
`executive-summary.md`	One-page business case and outcomes for sponsors/leadership.

PowerShell scripts

All scripts carry a `.SYNOPSIS` header with PURPOSE / HOW TO RUN / EXPECTED RESULTS and a NEUTRAL white-label `.NOTES`. Read-only and discovery scripts make no changes; the remediation script defaults to `-WhatIf` and requires `-Apply` to act, on **named objects only**.

Script	What it does
`scripts\INCIDENT-Discovery.ps1`	**Live incident snapshot (read-only).** Pulls current Microsoft 365 + Azure **Service Health**, recent **high-severity sign-ins / risk detections** from Entra, and an **Azure Resource Health** snapshot into one CSV to seed diagnosis.
`scripts\INCIDENT-Validation.ps1`	**Responder readiness (PASS/FAIL).** Verifies responders hold the required roles, can read the key logs (Service Health, sign-in, message trace, Azure Monitor), and that a Microsoft support plan is in place; emits PASS/FAIL per check for the §05 readiness gates.
`scripts\INCIDENT-Report.ps1`	**Incident metrics report.** Reads incident timeline data and computes **MTTA/MTTR** and per-severity counts/trends to CSV — the program dashboard that proves the runbook is working.
`scripts\INCIDENT-Remediation.ps1`	**Safe common fixes.** Performs scoped, well-understood remediations for a **named object** (e.g. revoke a user's sessions, re-enable a disabled mail-flow connector). Runs read-only / `-WhatIf` by default; requires `-Apply` to change anything.

Script	What it does
`scripts\INCIDENT-Set-ResponderRoles.ps1`	**Responder access setup.** Creates the `Responder Access Group – Identity – Response` Entra group and assigns the reader/scoped-admin roles responders need; supports `-WhatIf` preview before `-Apply`.

Hero image

File	Purpose
`INCIDENT-hero.png`	Listing/marketing hero image for the solution.

> ***Naming convention reminder:*** *the scripts and runbook share the object-naming standard – – (e.g.* `Responder Access Group – Identity – Response`*,* `Microsoft Escalation Sheet – Incident – Escalate`*) so outputs cross-reference cleanly across discovery, validation, remediation, and reporting.*

Appendix A — PowerShell Scripts

Included in the accompanying ZIP. Each script's header documents full usage.

INCIDENT-Discovery.ps1

INCIDENT-Remediation.ps1

INCIDENT-Report.ps1

INCIDENT-Set-ResponderRoles.ps1

INCIDENT-Validation.ps1